



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 3: THIẾT KẾ DỮ LIỆU (02)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Giải thích được các dạng chuẩn (1NF, 2NF, 3NF, BCNF) và tại sao cần chuẩn hóa.
2. **MT2.** Chuẩn hóa một ERD chưa chuẩn lên dạng 3NF.
3. **MT3.** Chuyển ERD mức Logical sang schema SQL (CREATE TABLE) với đầy đủ ràng buộc.
4. **MT4.** Thiết kế indexes phù hợp cho các truy vấn thường gặp.
5. **MT5.** Phân tích trade-off giữa chuẩn hóa và hiệu năng, biết khi nào nên denormalize.

TÀI LIỆU VÀ CÔNG CỤ CẦN THIẾT

- ❑ ERD mức Logical đã hoàn thiện từ buổi 1 (mỗi nhóm).
- ❑ PostgreSQL hoặc MySQL đã cài đặt trên máy.
- ❑ Công cụ quản lý DB: pgAdmin (PostgreSQL), MySQL Workbench, hoặc DBeaver.
- ❑ Editor SQL: có thể dùng VS Code với extension SQLTools.
- ❑ Template schema SQL mẫu (GV phát).

NỘI DUNG

1. Chuẩn hoá CSDL (Database Normalization)
2. Thiết kế mức vật lý (Physical Design)
3. Tối ưu hoá truy xuất dữ liệu
4. Ví dụ minh hoạ (Quản lý quán cafe)
5. Thực hành

1. Chuẩn hoá CSDL (Database Normalization)

1.1. Tại sao cần chuẩn hoá?

- ❑ Chuẩn hóa là quá trình tổ chức dữ liệu trong cơ sở dữ liệu để giảm thiểu dư thừa (redundancy) và tránh các bất thường (anomalies) khi thêm, sửa, xóa dữ liệu.
- ❑ Được E.F. Codd đề xuất vào đầu thập niên 1970, đây là nền tảng của thiết kế CSDL quan hệ.

1. Chuẩn hoá CSDL (Database Normalization)

1.1. Tại sao cần chuẩn hoá?

❑ Ví dụ về bất thường nếu KHÔNG chuẩn hóa: Giả sử bảng DON_HANG lưu trực tiếp thông tin khách hàng:

DON_HANG_KO_CHUAN

maDH	maKH	tenKH	sdtKH	maMon	tenMon	soLuong
DH001	KH01	Anh Tuan	0901234567	M01	Cafe sua	2
DH001	KH01	Anh Tuan	0901234567	M02	Croissant	1
DH002	KH02	Chi Minh	0912345678	M01	Cafe sua	1

1. Chuẩn hoá CSDL (Database Normalization)

1.1. Tại sao cần chuẩn hoá?

❑ Các vấn đề:

- ✓ **Dư thừa:** Thông tin khách hàng (tenKH, sdtKH) lặp lại mỗi dòng.
- ✓ **Bất thường cập nhật:** Nếu KH01 đổi số điện thoại, phải sửa nhiều dòng.
- ✓ **Bất thường xóa:** Xóa đơn hàng cuối cùng của KH02 → mất luôn thông tin KH02.
- ✓ **Bất thường thêm:** Không thể lưu thông tin khách hàng nếu chưa có đơn hàng.

DON_HANG_KO_CHUAN

maDH	maKH	tenKH	sdtKH	maMon	tenMon	soLuong
DH001	KH01	Anh Tuan	0901234567	M01	Cafe sua	2
DH001	KH01	Anh Tuan	0901234567	M02	Croissant	1
DH002	KH02	Chi Minh	0912345678	M01	Cafe sua	1

1. Chuẩn hoá CSDL (Database Normalization)

1.2. Dạng chuẩn 1 (First Normal Form – 1NF)

❑ Một bảng được gọi là ở dạng 1NF nếu:

- 1) Mỗi cột chỉ chứa giá trị nguyên tử (atomic) – không chia nhỏ được nữa.
- 2) Mỗi cột chỉ chứa một kiểu dữ liệu duy nhất.
- 3) Mỗi cột có tên duy nhất.
- 4) Thứ tự các hàng không quan trọng (không phụ thuộc vào vị trí).

1. Chuẩn hoá CSDL (Database Normalization)

1.2. Dạng chuẩn 1 (First Normal Form – 1NF)

❑ Ví dụ bảng vi phạm 1NF:

DON_HANG_VI_PHAM

maDH	ngayTao	danhSachMon
------	---------	-------------

DH001	10/11/25	Cafe sua (x2), Croissant (x1)
DH002	10/11/25	Cafe sua (x1)

1. Chuẩn hoá CSDL (Database Normalization)

1.2. Dạng chuẩn 1 (First Normal Form – 1NF)

❑ Chuyển về 1NF: Tách thành hai bảng riêng

DON_HANG		CHI_TIET_DON_HANG		
maDH	ngayTao	maDH	maMon	soLuong
DH001	10/11/25	DH001	M01	2
DH002	10/11/25	DH001	M02	1
		DH002	M01	1

1. Chuẩn hoá CSDL (Database Normalization)

1.3. Dạng chuẩn 2 (Second Normal Form – 2NF)

❑ Khái niệm phụ thuộc hàm (Functional Dependency):

- ✓ **Ký hiệu:** $X \rightarrow Y$ nghĩa là Y phụ thuộc hàm vào X.
- ✓ **Ví dụ:** $maKH \rightarrow tenKH$ (biết mã khách hàng thì xác định được tên).

❑ Một bảng ở dạng 2NF nếu:

- 1) Đã ở dạng 1NF.
- 2) Không có phụ thuộc hàm từng phần: Nếu Khóa chính của bảng là khóa tổ hợp (gồm từ 2 cột trở lên), thì tất cả các cột còn lại phải phụ thuộc vào toàn bộ khóa chính đó, chứ không được phép phụ thuộc vào chỉ một phần (một cột lẻ) của khóa.

1. Chuẩn hoá CSDL (Database Normalization)

1.3. Dạng chuẩn 2 (Second Normal Form – 2NF)

❑ Ví dụ vi phạm 2NF

CHI_TIET_DON_HANG_VI_PHAM

maDH	maMon	soLuong	tenMon	donGia
DH001	M01	2	Cafe sua	30000
DH001	M02	1	Croissant	45000

- ✓ **Khóa chính:** (maDH, maMon)
- ✓ **Vấn đề:** tenMon và donGia chỉ phụ thuộc vào maMon, không phụ thuộc vào maDH → vi phạm 2NF.

1. Chuẩn hoá CSDL (Database Normalization)

1.3. Dạng chuẩn 2 (Second Normal Form – 2NF)

❑ Chuyển về 2NF: Tách thành hai bảng riêng

CHI_TIET_DON_HANG

maDH	maMon	soLuong
-----+-----+-----		
DH001	M01	2
DH001	M02	1

MON

maMon	tenMon	donGia
-----+-----+-----		
M01	Cafe sua	30000
M02	Croissant	45000

1. Chuẩn hoá CSDL (Database Normalization)

1.4. Dạng chuẩn 3 (Third Normal Form – 3NF)

□ Một bảng ở dạng 3NF nếu:

- 1) Đã ở dạng 2NF.
- 2) Không có phụ thuộc bắc cầu (transitive dependency): các cột không phải là khóa không được phép suy ra lẫn nhau. Chúng chỉ được phép phụ thuộc trực tiếp vào Khóa chính mà thôi.

1. Chuẩn hoá CSDL (Database Normalization)

1.4. Dạng chuẩn 3 (Third Normal Form – 3NF)

❑ Ví dụ vi phạm 3NF

DON_HANG_VI_PHAM_3NF

maDH	maKH	tenKH	thanhPho	quocGia
-----+-----+-----+-----+-----				
DH001	KH01	Anh Tuan	Ho Chi Minh	Vietnam
DH002	KH02	Chi Minh	Ha Noi	Vietnam

- ✓ Phụ thuộc hàm: $maDH \rightarrow maKH$; $maKH \rightarrow tenKH, thanhPho$; $thanhPho \rightarrow quocGia$ (phụ thuộc bắc cầu!)
- ✓ Vấn đề: $quocGia$ phụ thuộc vào $thanhPho$, mà $thanhPho$ không phải khóa $\rightarrow$ vi phạm 3NF

1. Chuẩn hoá CSDL (Database Normalization)

1.4. Dạng chuẩn 3 (Third Normal Form – 3NF)

❑ Chuyển về chuẩn 3NF: Tách bảng

DON_HANG		KHACH_HANG			THANH_PHO		
maDH	maKH	maKH	tenKH	maTP	maTP	tenTP	quocGia
-----+-----		-----+-----+-----			-----+-----+-----		
DH001	KH01	KH01	Anh Tuan	TP01	TP01	HCM	Vietnam
DH002	KH02	KH02	Chi Minh	TP02	TP02	HN	Vietnam

1. Chuẩn hoá CSDL (Database Normalization)

1.5. Dạng chuẩn Boyce-Codd (BCNF)

❑ Một bảng ở dạng BCNF nếu:

- 1) Đã ở dạng 3NF.
- 2) Với mọi phụ thuộc hàm $X \rightarrow Y$, X phải là siêu khóa (superkey).
- 3) BCNF chặt chẽ hơn 3NF một chút. Trong hầu hết trường hợp, 3NF và BCNF trùng nhau

❑ Đối với hầu hết ứng dụng nghiệp vụ, đạt 3NF là đủ. BCNF (Boyce-Codd Normal Form) và các dạng cao hơn (4NF, 5NF) chỉ cần trong các hệ thống phức tạp đặc biệt.

1. Chuẩn hoá CSDL (Database Normalization)

1.6. Khi nào không cần chuẩn hoá? (Denormalization)

- ❑ Trong một số tình huống, denormalization (phi chuẩn hóa có chủ ý) cải thiện hiệu năng:
 - ✓ Truy vấn đọc nhiều hơn ghi: Nếu hệ thống có 95% là đọc (read-heavy), dư thừa một ít dữ liệu để giảm JOIN có thể nhanh hơn.
 - ✓ Báo cáo và phân tích: Data warehouse thường denormalize để tăng tốc query phức tạp.
 - ✓ Caching và materialized views: Lưu kết quả tính toán sẵn thay vì tính lại mỗi lần.
 - ✓ NoSQL databases: Các DB dạng document (MongoDB) khuyến khích nhúng dữ liệu liên quan vào cùng document

2. Thiết kế mức vật lý (Physical Design)

2.1. Từ Logical sang Physical

- ❑ Ở mức Logical, chúng ta có ERD với các thực thể, thuộc tính, quan hệ, nhưng chưa cụ thể hóa. Mức Physical là bước triển khai lên hệ quản trị cụ thể (PostgreSQL, MySQL...) với:
 - ✓ Kiểu dữ liệu cụ thể (INT, VARCHAR, DECIMAL, TIMESTAMP...)
 - ✓ Ràng buộc toàn vẹn (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL)
 - ✓ Giá trị mặc định (DEFAULT)
 - ✓ Indexes để tăng tốc truy vấn
 - ✓ Triggers, stored procedures (nếu cần)

2. Thiết kế mức vật lý (Physical Design)

2.2. Chọn kiểu dữ liệu phù hợp

- ❑ Chọn kiểu dữ liệu nhỏ nhất đủ dùng: INT thay vì BIGINT nếu không cần số lớn.
- ❑ Ưu tiên kiểu số cho PK: tăng tự động → nhanh hơn khi JOIN.
- ❑ VARCHAR(n) với n hợp lý: KHÔNG dùng VARCHAR(255) cho mọi trường text.
- ❑ DECIMAL cho tiền tệ: tránh làm tròn sai với FLOAT.
- ❑ TIMESTAMP/DATETIME cho thời gian: có múi giờ nếu cần (TIMESTAMPTZ trong PostgreSQL).

2. Thiết kế mức vật lý (Physical Design)

2.2. Chọn kiểu dữ liệu phù hợp

Mục đích	PostgreSQL	MySQL	Ghi chú
Số nguyên nhỏ	SMALLINT	SMALLINT	16-bit, -32768 đến 32767
Số nguyên	INTEGER	INT	32-bit, $\sim \pm 2$ tỷ
Số nguyên lớn	BIGINT	BIGINT	64-bit
Số thập phân	NUMERIC(p,s)	DECIMAL(p,s)	Chính xác cho tiền
Số thực	REAL, DOUBLE	FLOAT, DOUBLE	Nhanh nhưng không chính xác
Chuỗi ký tự	VARCHAR(n)	VARCHAR(n)	n: độ dài tối đa
Text dài	TEXT	TEXT	Không giới hạn (Postgre)
Ngày giờ	TIMESTAMP	DATETIME	MySQL: không múi giờ
Ngày giờ + TZ	TIMESTAMPTZ	TIMESTAMP	PostgreSQL tốt hơn
Boolean	BOOLEAN	TINYINT(1)	MySQL dùng 0/1
JSON	JSON, JSONB	JSON	JSONB (PG) nhanh hơn

2. Thiết kế mức vật lý (Physical Design)

2.3. Các ràng buộc toàn vẹn (Integrity Constraints)

❑ PRIMARY KEY (Khóa chính)

- ✓ Xác định duy nhất mỗi bản ghi. Tự động tạo UNIQUE + NOT NULL

```
CREATE TABLE KHACH_HANG (maKH VARCHAR(20) PRIMARY KEY, hoTen  
VARCHAR(100) NOT NULL, soDienThoai VARCHAR(15));
```

- ✓ Hoặc dùng khóa tổ hợp (composite):

```
CREATE TABLE CHI_TIET_DON_HANG (maDH VARCHAR(30), maMon  
VARCHAR(20), soLuong INT NOT NULL, PRIMARY KEY (maDH, maMon));
```

2. Thiết kế mức vật lý (Physical Design)

2.3. Các ràng buộc toàn vẹn (Integrity Constraints)

❑ FOREIGN KEY (Khóa ngoại)

Đảm bảo tính tham chiếu (referential integrity). Giá trị FK phải tồn tại trong bảng được tham chiếu.

```
CREATE TABLE DON_HANG (  
    maDH VARCHAR(30) PRIMARY KEY,  
    maKH VARCHAR(20),  
    FOREIGN KEY (maKH) REFERENCES KHACH_HANG (maKH)  
    ON DELETE RESTRICT      -- Không cho xóa KH nếu còn đơn hàng  
    ON UPDATE CASCADE       -- Cập nhật maKH tự động nếu KH đổi mã  
);
```

2. Thiết kế mức vật lý (Physical Design)

2.3. Các ràng buộc toàn vẹn (Integrity Constraints)

❑ FOREIGN KEY (Khóa ngoại)

Các tùy chọn ON DELETE / ON UPDATE:

Tùy chọn	Ý nghĩa
CASCADE	Tự động xóa/cập nhật bản ghi con khi cha thay đổi
RESTRICT / NO ACTION	Ngăn không cho xóa/cập nhật nếu còn bản ghi con
SET NULL	Đặt FK thành NULL khi bản ghi cha bị xóa
SET DEFAULT	Đặt FK về giá trị mặc định

2. Thiết kế mức vật lý (Physical Design)

2.3. Các ràng buộc toàn vẹn (Integrity Constraints)

❑ UNIQUE

Đảm bảo giá trị không trùng lặp (nhưng cho phép NULL).

```
CREATE TABLE TAI_KHOAN (  
    username VARCHAR(50) PRIMARY KEY,  
    email VARCHAR(100) UNIQUE,  
    passwordHash VARCHAR(255) NOT NULL  
);
```

2. Thiết kế mức vật lý (Physical Design)

2.3. Các ràng buộc toàn vẹn (Integrity Constraints)

❑ NOT NULL và DEFAULT

```
CREATE TABLE DON_HANG (  
    maDH VARCHAR(30) PRIMARY KEY,  
    ngayTao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    trangThai VARCHAR(20) DEFAULT 'DA_TAO',  
    tongTien DECIMAL(10,2) NOT NULL DEFAULT 0  
);
```

2. Thiết kế mức vật lý (Physical Design)

2.4. Triggers và Stored Procedures

❑ **Trigger là đoạn mã tự động chạy khi có sự kiện INSERT, UPDATE, DELETE trên bảng. Dùng để:**

- ✓ Tự động cập nhật dữ liệu liên quan
- ✓ Ghi log thay đổi (audit trail)
- ✓ Kiểm tra điều kiện nghiệp vụ phức tạp

2. Thiết kế mức vật lý (Physical Design)

2.4. Triggers và Stored Procedures

❑ Ví dụ: Tự động cập nhật tongTien khi thêm chi tiết đơn hàng

```
CREATE OR REPLACE FUNCTION update_tongTien() RETURNS TRIGGER AS $$
BEGIN
    UPDATE DON_HANG    SET tongTien = (
        SELECT SUM(soLuong * donGiaTaiThoiDiem)
        FROM CHI_TIET_DON_HANG
        WHERE maDH = NEW.maDH
    )
    WHERE maDH = NEW.maDH;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_update_tongTien AFTER INSERT OR UPDATE ON
CHI_TIET_DON_HANG FOR EACH ROW EXECUTE FUNCTION update_tongTien();
```

2. Thiết kế mức vật lý (Physical Design)

2.4. Triggers và Stored Procedures

❑ **Stored Procedure:** Thủ tục lưu trữ là đoạn mã SQL có thể tái sử dụng, thường dùng cho logic nghiệp vụ phức tạp.

```
CREATE OR REPLACE PROCEDURE tao_don_hang_moi(  
    p_maKH VARCHAR,  
    p_maNV VARCHAR,  
    OUT p_maDH VARCHAR ) LANGUAGE plpgsql AS  
$$ BEGIN    p_maDH := 'ORD-' || TO_CHAR(NOW(), 'YYYYMMDD') || '-' ||  
LPAD(nextval('seq_donhang')::TEXT, 3, '0');  
    INSERT INTO DON_HANG (maDH, maKH, maNV, ngayTao, trangThai,  
tongTien)  
    VALUES (p_maDH, p_maKH, p_maNV, NOW(), 'DA_TAO', 0);  
END; $$;
```

3. Tối ưu hoá truy xuất dữ liệu

3.1. Indexes (Chỉ mục)

- ❑ Index là cấu trúc dữ liệu phụ giúp tăng tốc độ truy vấn dữ liệu. Tương tự như mục lục của sách: thay vì đọc từng trang, tra mục lục để nhảy thẳng đến trang cần tìm.
- ❑ Các loại index phổ biến:
 - ✓ **B-Tree Index**: mặc định, phù hợp cho hầu hết truy vấn (=, <, >, BETWEEN, LIKE 'abc%').
 - ✓ **Hash Index**: chỉ cho phép tìm kiếm bằng (=), rất nhanh nhưng hạn chế.
 - ✓ **GiST/GIN Index**: cho full-text search, array, JSON (PostgreSQL).
 - ✓ **Partial Index**: chỉ index một phần dữ liệu thỏa điều kiện.
 - ✓ **Composite Index**: index trên nhiều cột (a, b, c).

3. Tối ưu hoá truy xuất dữ liệu

3.1. Indexes (Chỉ mục)

❑ Khi nào nên tạo index?

- ✓ Cột thường xuất hiện trong WHERE, JOIN, ORDER BY, GROUP BY.
- ✓ Cột có tính phân biệt cao (cardinality cao) – nhiều giá trị khác nhau.
- ✓ Bảng lớn (hàng chục ngàn bản ghi trở lên).
- ✓ Truy vấn đọc nhiều hơn ghi.

3. Tối ưu hoá truy xuất dữ liệu

3.1. Indexes (Chỉ mục)

❑ Khi nào **KHÔNG** nên tạo index?

- ✓ Cột có ít giá trị khác nhau (vd: giới tính TRUE/FALSE).
- ✓ Bảng nhỏ (< 1000 bản ghi) – full scan nhanh hơn.
- ✓ Cột thường xuyên bị UPDATE – index làm chậm việc ghi.
- ✓ Over-indexing: quá nhiều index làm tốn bộ nhớ và chậm INSERT/UPDATE.

3. Tối ưu hoá truy xuất dữ liệu

3.1. Indexes (Chỉ mục)

❑ Ví dụ tạo index:

```
-- Index đơn
CREATE INDEX idx_donhang_ngaytao ON DON_HANG(ngayTao);

-- Composite index
CREATE INDEX idx_donhang_kh_ngay ON DON_HANG(maKH, ngayTao);

-- Partial index (chỉ index đơn hàng đang hoạt động)
CREATE INDEX idx_donhang_active ON DON_HANG(maDH) WHERE trangThai
NOT IN ('DA_HUY', 'HOAN_THANH');

-- Unique index
CREATE UNIQUE INDEX idx_taikhoan_email ON TAI_KHOAN(email);
```

3. Tối ưu hoá truy xuất dữ liệu

3.1. Indexes (Chỉ mục)

- ❑ **Composite Index và Left-Most Prefix Rule:** Khi tạo composite index (a, b, c), index này có thể dùng cho:
 - ✓ WHERE a = ?
 - ✓ WHERE a = ? AND b = ?
 - ✓ WHERE a = ? AND b = ? AND c = ?
 - ✓ Nhưng KHÔNG dùng được cho:
 - ✓ WHERE b = ? (thiếu a)
 - ✓ WHERE c = ? (thiếu a và b)
 - ✓ Quy tắc: phải dùng từ cột trái nhất.

3. Tối ưu hoá truy xuất dữ liệu

3.2. Query Optimization (Tối ưu câu truy vấn)

❑ Dùng EXPLAIN để xem cách DB thực thi câu truy vấn:

```
EXPLAIN ANALYZE SELECT dh.maDH, kh.hoTen, SUM(ct.soLuong *  
ct.donGiaTaiThoiDiem) AS tongTien FROM DON_HANG dh JOIN KHACH_HANG  
kh ON dh.maKH = kh.maKH JOIN CHI_TIET_DON_HANG ct ON dh.maDH =  
ct.maDH WHERE dh.ngayTao >= '2025-01-01' GROUP BY dh.maDH, kh.hoTen;
```

❑ Kết quả EXPLAIN cho biết:

- ✓ Loại scan: Seq Scan (full table scan) vs Index Scan
- ✓ Cost: chi phí ước lượng
- ✓ Rows: số hàng ước tính
- ✓ Actual time: thời gian thực tế (nếu dùng ANALYZE)

3. Tối ưu hoá truy xuất dữ liệu

3.2. Query Optimization (Tối ưu câu truy vấn)

Các kỹ thuật tối ưu query

1) **Tránh SELECT *:** Chỉ lấy các cột cần thiết:

```
-- Không nên  
SELECT * FROM DON_HANG WHERE maKH = 'KH001';  
  
-- Nên  
SELECT maDH, ngayTao, tongTien FROM DON_HANG WHERE maKH = 'KH001';
```

2) **Sử dụng LIMIT:** Nếu chỉ cần một vài bản ghi đầu, thêm LIMIT

```
SELECT maDH, tongTien FROM DON_HANG ORDER BY ngayTao DESC LIMIT 10;
```

3. Tối ưu hoá truy xuất dữ liệu

3.2. Query Optimization (Tối ưu câu truy vấn)

Các kỹ thuật tối ưu query

3) Tránh hàm trong WHERE: Hàm trên cột làm index không dùng được

```
-- Không nên (không dùng index)
SELECT * FROM DON_HANG WHERE YEAR(ngayTao) = 2025;

-- Nên (dùng index)
SELECT * FROM DON_HANG WHERE ngayTao >= '2025-01-01' AND ngayTao <
'2026-01-01';
```

3. Tối ưu hoá truy xuất dữ liệu

3.2. Query Optimization (Tối ưu câu truy vấn)

Các kỹ thuật tối ưu query

4) JOIN thay vì Subquery: Trong nhiều trường hợp, JOIN nhanh hơn subquery

```
-- Chậm hơn
SELECT * FROM DON_HANG WHERE maKH IN (SELECT maKH FROM KHACH_HANG
WHERE thanhPho = 'HCM');

-- Nhanh hơn
SELECT dh.* FROM DON_HANG dh JOIN KHACH_HANG kh ON dh.maKH = kh.maKH
WHERE kh.thanhPho = 'HCM';
```

3. Tối ưu hoá truy xuất dữ liệu

3.2. Query Optimization (Tối ưu câu truy vấn)

Các kỹ thuật tối ưu query

5) Sử dụng **UNION ALL** thay vì **UNION**:

- ☐ **UNION** tự động loại bỏ trùng (**DISTINCT**) nên chậm hơn.
- ☐ Nếu biết chắc không trùng, dùng **UNION ALL**.

3. Tối ưu hoá truy xuất dữ liệu

3.3. Materialized Views và Caching

- ❑ **Materialized View:** Lưu kết quả của một query phức tạp thành bảng vật lý. Dùng cho báo cáo hoặc dashboard cần refresh định kỳ.

```
CREATE MATERIALIZED VIEW v_doanhthu_thang AS
SELECT
    DATE_TRUNC('month', ngayTao) AS thang,
    SUM(tongTien) AS doanhThu,
    COUNT(maDH) AS soDonHang
FROM DON_HANG
WHERE trangThai = 'HOAN_THANH'
GROUP BY DATE_TRUNC('month', ngayTao);

-- Refresh khi cần
REFRESH MATERIALIZED VIEW v_doanhthu_thang;
```

3. Tối ưu hoá truy xuất dữ liệu

3.3. Materialized Views và Caching

❑ **Application-level Caching:** Dùng Redis hoặc Memcached để cache kết quả query phổ biến:

- ✓ Cache danh sách menu món (ít thay đổi, đọc nhiều)
- ✓ Cache thông tin khách hàng VIP
- ✓ Cache kết quả tìm kiếm phổ biến

TTL (Time To Live) hợp lý: 5-60 phút tùy dữ liệu.

3. Tối ưu hoá truy xuất dữ liệu

3.4. Trade-offs và Best Practices

Kỹ thuật	Ưu điểm	Nhược điểm	Khi nào dùng
Index nhiều	Truy vấn nhanh	INSERT/UPDATE chậm, tốn RAM	Read-heavy systems
Denormalization	Giảm JOIN	Dư thừa, khó maintain	Data warehouse, reporting
Materialized View	Báo cáo nhanh	Dữ liệu không real-time	Dashboard, analytics
Partitioning	Query trên bảng lớn nhanh	Phức tạp setup	Bảng > 10 triệu rows
Caching (Redis)	Cực nhanh (RAM)	Phức tạp invalidation	Dữ liệu ít thay đổi

3. Tối ưu hoá truy xuất dữ liệu

3.4. Trade-offs và Best Practices

❑ Nguyên tắc chung:

- ✓ Measure before optimize: dùng EXPLAIN, pgAdmin stats, slow query log.
- ✓ Optimize the bottleneck: tìm query chậm nhất, tối ưu nó trước.
- ✓ Don't over-engineer: giải pháp đơn giản thường đủ.
- ✓ Test with production-like data: test trên 1000 rows khác với 10 triệu rows.

THỰC HÀNH

1. Cài đặt PostgreSQL

2. Cài đặt SQL GUI: Chọn một trong các phần mềm sau

- 1) **pgAdmin 4** (Chính chủ, đầy đủ tính năng nhất)
- 2) **DBeaver** (Đa năng, khuyên dùng nếu làm việc với nhiều loại DB)
- 3) **TablePlus** (UI đẹp, mượt nhất trên MacOS)

3. Chuẩn hoá ERD: Kiểm tra các ERD của nhóm có vi phạm 1NF, 2NF, 3NF không? Nếu có thì chuẩn hoá và update lại.

4. Viết schema SQL: Chuyển ERD đã chuẩn hóa thành schema SQL hoàn chỉnh với đầy đủ ràng buộc và tạo bảng trong PostgreSQL.

5. Thiết kế indexes: Xác định các truy vấn thường gặp trong BTL và thiết kế indexes phù hợp.

- ❑ Liệt kê 5-10 câu truy vấn quan trọng nhất của hệ thống:
 - ✓ Vd: "Lấy tất cả đơn hàng của khách hàng X"
 - ✓ Vd: "Tìm món bán chạy nhất tháng này"
 - ✓ Vd: "Báo cáo doanh thu theo ngày"
- ❑ Với mỗi query, xác định các cột xuất hiện trong WHERE, JOIN, ORDER BY, GROUP BY. Viết CREATE INDEX cho các cột đó.
- ❑ Ưu tiên index cho: Foreign keys; Cột thời gian (ngayTao, ngayCapNhat); Cột trạng thái (trangThai)
- ❑ Test bằng EXPLAIN

THỰC HÀNH

Tiêu chí	Mô tả
☐ Đầy đủ bảng	Tất cả thực thể trong ERD đều có CREATE TABLE
☐ PK đúng	Mỗi bảng có PRIMARY KEY
☐ FK đúng	Quan hệ 1-N, N-N được biểu diễn bằng FOREIGN KEY
☐ Kiểu dữ liệu	Phù hợp: DECIMAL cho tiền, TIMESTAMP cho thời gian
☐ NOT NULL	Các trường bắt buộc có NOT NULL
☐ CHECK	Ràng buộc giá trị hợp lệ (vd: gia > 0, trạngThai IN ...)
☐ ON DELETE	Xử lý đúng khi xóa bản ghi cha